On Interplanetary and Relativistic Distributed Computing

Siddhartha Jayanti
సిద్ధార్థ విశ్వేశ్వర జయంతి
Dartmouth College
Hanover, NH, USA
siddhartha.visveswara.jayanti@dartmouth.edu

Abstract

Interplanetary distributed systems, such as the Interplanetary Internet, and the Global Positioning System (GPS) are subject to the effects of Einstein's theory of relativity. In this paper, we study *relativistic distributed systems*, which are subject to the *relativity of simultaneity*. We formulate a unified computational model for relativistic and classical distributed systems and study the relationship between properties of distributed algorithms deployed on the two types of systems. Classical executions are totally ordered in time, whereas the steps of a relativistic execution are only partially ordered by the relation of relativistic causality. We relate these two physics-dependent execution types through a third—purely mathematical—notion of a *computational execution*, which partially orders steps by the relation of computational causality. We relate relativistic, classical, and computational executions of distributed algorithms through a central theorem, which states that the following are equivalent for any distributed algorithm $\mathcal{A}$: (1) $\mathcal{A}$ satisfies a property $\mathcal{P}$ classically; (2) every relativistic execution of $\mathcal{A}$ satisfies $\mathcal{P}$ in the reference frame of every observer; and (3) every total ordering of every computational execution of $\mathcal{A}$ satisfies $\mathcal{P}$. As a direct consequence, we prove the equivalence of the standard, relativistic, and computational formulations of linearizability. Our results show that a host of algorithms originally designed for classical distributed systems will behave consistently when deployed in relativistic, interplanetary distributed systems.

CCS Concepts

• **Theory of computation** → **Distributed computing models**; **Distributed algorithms**; **Data structures design and analysis**.

Keywords

relativistic distributed computing, interplanetary internet, relativistic linearizability, computational linearizability, causality

ACM Reference Format:
Siddhartha Jayanti. 2025. On Interplanetary and Relativistic Distributed Computing. In *ACM Symposium on Principles of Distributed Computing (PODC '25), June 16–20, 2025, Huatulco, Mexico.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3732772.3733563

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '25, Huatulco, Mexico
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1885-4/2025/06
https://doi.org/10.1145/3732772.3733563

पूर्णः कुम्भोऽधि काल आहितस्तं वै पश्यामो बहुधा नु सन्तः।
Time is placed in the cosmos. We observe it from various perspectives.

अथर्ववेदः १९.५३.३
Atharvaveda 19.53.3

1 Introduction

Distributed systems are embedded in the physical universe, thus the physical nature of space and time are material to the understanding of distributed computation. It is common to design distributed algorithms under the classical assumption that the events that transpire in a system can be totally ordered by a universally accepted notion of time, and to analyze distributed systems by reasoning about the evolution of the global state of the system over time. While this assumption of *absolute time* is in line with everyday experience and classical physics, it does not hold for systems in which the effects of Einstein's theory of relativity [10] are too large to ignore, such as the Global Positioning System (GPS) and the Interplanetary Internet [16] of satellites, space shuttles, and space probes that span our solar system. In fact, the GPS technology that empowers our everyday travels relies on at least two important relativistic corrections—one to counter the special relativistic effect of kinetic time dilation and another to counter the general relativistic effect of gravitational time dilation [3].

In this work, we study *relativistic distributed computing*—the design and analysis of distributed algorithms deployed on systems embedded in a relativistic physics. Our main result is a *central theorem* which relates the properties of relativistic and classical executions of asynchronous distributed algorithms, showing that *an algorithm $\mathcal{A}$ satisfies a property $\mathcal{P}$ classically, if and only if $\mathcal{P}$ is satisfied in the perception of every observer—regardless of his frame of reference—in every relativistic execution of $\mathcal{A}$.*

As an illustration, consider the classic correctness property for distributed shared objects—*linearizability* [15]. This gold standard states that an implementation of a shared object—such as a database, key-value store, or an object of any other type—should behave as if its operations take effect instantaneously in the time interval of their execution.

In 2014, Gilbert and Golab [12] observed that the *relativity of simultaneity* [11]—which asserts that different observers may see the events of the system transpire in different orders—undermines the definition of linearizability. Like most classic properties, linearizability is defined under the assumption that computational events can be totally ordered, whereas events in a relativistic distributed system are, in general, only partially ordered. To put relativistic distributed system design on a strong foundation nevertheless, they proposed *relativistic linearizability* as a strong consistency condition for such systems. An implementation of a shared object

is relativistically linearizable, if every observer in the universe—regardless of his frame of reference—agrees that the object is linearizable according to his local clock. While Gilbert and Golab proved that relativistic linearizability has some desirable properties, such as composability (a.k.a. locality), and presented some tools to reason about relativistic linearizability, little was known about relativistically linearizable *algorithms*. In fact, to our knowledge, there are no previous proofs of any algorithms being relativistically linearizable. As a result, an understanding of which algorithms are safe to deploy in interplanetary systems has been left wanting. An immediate consequence of our central theorem is an *equivalence theorem* that ameliorates this state of affairs, by guaranteeing that: *every linearizable algorithm is relativistically linearizable and every relativistically linearizable algorithm is linearizable.* Consequently, the host of asynchronous linearizable algorithms that the distributed computing community has discovered for classical distributed systems will remain strongly consistent when deployed in relativistic, interplanetary distributed systems.

1.1 Challenges and Our Approach

Reasoning about relativistic distributed systems is challenging, since it requires analysis that combines distributed computational reasoning with physics. To facilitate such analysis, we abstract some main ideas from physics that are pertinent to understanding distributed systems, and summarize the relevant notions of classical and relativistic physics into a handful of postulates. These postulates allow for the mathematical abstraction of physical executions of distributed algorithms.

Even with appropriate abstractions, relativity is counter-intuitive. In general, events of the system are only partially ordered by the *relativistic (causal) order*. While any given observer *perceives* events as occurring in a total order according to the *local time* of his *frame of reference*, the order of the events that are not related by the relativistic order can vary widely between observers. Consequently, classical models that model executions as sequences of global states (e.g., [27]) cannot be used, since different processors have equally valid, yet different perceptions of the global state. In order to relate classical and relativistic executions, we present a unified theoretical model of distributed algorithms that is based solely on *local steps*. Our model draws inspiration from Gilbert and Golab's model of relativistic histories [12]. In essence, a classical execution is a (valid and physically permissible) collection of steps that is endowed with a total ordering corresponding to the chronological order of absolute time; whereas a relativistic execution is endowed with a partial order that corresponds to relativistic causality. Our model is general enough to model both message passing and shared-memory distributed systems in both classical and relativistic physics.

Directly reasoning about properties of relativistic executions can be difficult, since reasoning about a single relativistic execution amounts to reasoning about how infinitely many observers perceive it. For example, proving that an execution is relativistically linearizable requires proving that *every* possible observer's perception of that execution is linearizable—including external observers that do not correspond to processors in the distributed system. To make such reasoning tractable, we define the notion of a *computational execution* of an algorithm. Unlike classical and relativistic executions—which are physics dependent—computational executions are purely mathematical abstractions that partially order computational steps according to Lamport's [24] computational causal precedence order. In our analysis, we prove that classical and relativistic executions can be related effectively through the mechanism of computational executions. Building on this relationship, we prove the following *Central Theorem* that states a tight connection between the properties of classical, relativistic, and computational executions: *a property $\mathcal{P}$ is satisfied by all classical executions of an algorithm $\mathcal{A}$, if and only if $\mathcal{P}$ is satisfied in the perception of every observer—regardless of his frame of reference—in every relativistic execution of $\mathcal{A}$, if and only if $\mathcal{P}$ is satisfied in every total ordering of every computational execution of $\mathcal{A}$.* The Central Theorem is powerful, since it states a general relationship between properties of classical executions and the properties perceived by observers in relativistic executions. We capitalize on this relationship to derive the equivalence of classical and relativistic linearizability. In fact, we also show that both of these notions are equivalent to a novel, physics-independent, notion of *computational linearizability*.

1.2 Our Contributions

Our paper makes the following contributions.

(1) **Modeling:** We present a unified model of classical and relativistic distributed systems for message-passing and shared-memory. Our modeling includes: abstracting the relevant physical postulates to reason about distributed systems in classical and relativistic physics, and defining classical, relativistic, and computational executions of distributed algorithms.
(2) **Central Theorem:** Let $\mathcal{A}$ be any distributed algorithm and $\mathcal{P}$ be any property of totally ordered executions. Then the following three statements are equivalent:
   - $\mathcal{P}$ is satisfied by all classical executions of $\mathcal{A}$.
   - $\mathcal{P}$ is satisfied in the reference frame of every observer of every relativistic execution of $\mathcal{A}$.
   - $\mathcal{P}$ is satisfied by all total orderings of computational executions of $\mathcal{A}$.
(3) **Equivalence Theorem:** Let $\mathcal{A}$ be any distributed implementation of a shared object, then the following three statements are equivalent:
   - $\mathcal{A}$ is linearizable.
   - $\mathcal{A}$ is relativistically linearizable.
   - $\mathcal{A}$ is computationally linearizable.

Our results imply that a host of algorithms that the distributed computing community has discovered can be safely deployed in relativistic, interplanetary distributed systems.

2 Related Work

Considerations of relativity have been sparse in the distributed computing literature, however there are notable works that discuss relativity in distributed systems. Lamport briefly considers special relativity in his papers on time [24] and communication [25, 26]. Gilbert and Golab initiated the study of linearizability in relativistic distributed systems [12]; we elaborate on their work later in this section. The Global Positioning System (GPS) is a concrete example of a distributed system which experiences and corrects for

relativistic effects [3]. We expect relativistic distributed computing to burgeon with the expansion of the Interplanetary Internet [16] and increased interest in space exploration [8, 29], including attempts to establish extraterrestrial human colonies [28].

Linearizability [15] is the gold standard for classical data structure correctness since it guarantees: (1) the *appearance of atomic operations*, (2) *adherence to real-time ordering*, and (3) *composability (i.e., locality)*. Due to these important properties, variants of linearizability are used to define correctness across types of distributed systems and algorithms including: *strong linearizability* [13] for real-time determination of linearization; *strict linearizability* [1] for crash-prone systems; *durable* [18] and *recoverable* [4, 6, 19] linearizability for crash-restart systems; and *direct linearizability* [5] for snapshottable systems.

Computational causality was defined by Lamport through his *happens-before* relation [24], which determines if one computational event *could have* affected another. A shared object is *causally consistent* [2] if causally related operations appear to take effect in the causal precedence order to all processes, but other operations may appear to take effect in different orders to different processes. In this sense, causal consistency has a spurious likeness to frames of reference in relativity, leading some authors to use the term *relativistic* when discussing classical causally consistent programming [17, 30]. Causal consistency is not composable and does not guarantee adherence to real-time ordering.

Gilbert and Golab [12] initiated the study of *relativistic linearizability* with the goal of defining a linearizability-like condition for relativistic distributed systems. They considered three possible candidate definitions, which they termed, R_1, R_2, and R_3. All three definitions stem from the key observation that events, which are only partially ordered in a relativistic history, are totally ordered in any given reference frame. Thus, observing a relativistic history in a specific reference frame yields a totally ordered classical history for which linearizability is well-defined. Consequently, they model reference frames as total orderings. They define a shared object implementation to be: R_1*-linearizable*, if every relativistic execution of that implementation appears linearizable in *some* total ordering of the partially ordered events; R_2*-linearizable*, if every relativistic execution appears linearizable in *every* total ordering; and R_3*-linearizable*, if every relativistic execution appears linearizable in *every* total ordering *with the additional constraint* that all the total orderings also agree on a linearization. They prove that of these three definitions, only R_2-linearizability is composable. Along with their investigation of composability, Gilbert and Golab presented some tools to reason about relativistic linearizability. For example, they give conditions under which R_3-linearizability can be inferred from R_2-linearizability and similarly R_2-linearizability inferred from R_1-linearizability. To our knowledge, there is no previous proof that any *algorithm* satisfies relativistic linearizability.

In this paper, we define a shared object implementation to be *relativistically linearizable*, if every relativistic execution of that implementation appears linearizable in the reference frame of every observer. Our definition differs slightly from R_2-linearizability, since it restricts focus to the total orderings which correspond to actual frames of reference. We prove that *all* asynchronous linearizable algorithms are relativistically linearizable for either definition.

3 Classical & Relativistic Physics and Distributed Systems

A *distributed system* consists of a set of asynchronous processors, Π, that communicate by passing messages. The term *distributed* indicates that the processors are separated in space, possibly far away from each other and in relative motion with respect to each other. The processors are *asynchronous*, in the sense that they may spend arbitrary and variable times between computational steps. They may even fail by *crashing*, i.e., cease to take steps in the middle of an execution. Message transmission is also *asynchronous*, meaning that messages may take arbitrarily long to reach their intended recipient; they may reach out of order, and they may even be dropped entirely.

We model an atomic shared-object (e.g., a shared word) as a processor, ω, which: stores the state of the shared object, receives messages requesting it to perform operations $op(arg)$ on behalf of some processor π, and reacts to such messages by applying $op(arg)$ to the stored object state and sending the response back to π. A system in which all messages are such requests-to and responses-from atomic shared-objects is a pure shared-memory system.

A distributed system can be embedded in either a *classical*, i.e. Newtonian, or *relativistic*, i.e. Einsteinian, physics. The former is called a *classical distributed system* while the latter is called a *relativistic distributed system*. In both cases, processors are physically modeled as idealized *point-like* bodies. We model relevant aspects of these systems below.

3.1 Classical Distributed Systems

Classical distributed systems are the standard object of study in the distributed computing literature. In classical physics, time is *absolute*. Thus, we identify each computational event e that occurs in the system with the time t at which it occurs. To stay consistent with the literature, we make the standard assumption [24, 27] that computational events occur at distinct times, and thus that they can be totally ordered, *chronologically*. We summarize this discussion in the form of the following *classical postulates*:

- CP1: Events of interest occur at distinct times, which are totally chronologically ordered by $<$.
- CP2: Transmissions can travel arbitrarily fast, but, they must be oriented forwards in time. So, it is physically permissible for a transmission sent at t to be received at t' if and only if $t < t'$.

3.2 Relativistic Distributed Systems

Relativistic distributed systems are subject to the effects of relativity theory, such as time-dilation, length-contraction, and relativity of simultaneity [11]. In particular, each processor $\pi \in \Pi$ may be in a different *(observational) reference frame*. *Events* of interest happen instantaneously at single points in four-dimensional *spacetime*—denoted by the set $\mathcal{E}$. In any particular reference frame R, an event e's spacetime coordinates can be expressed as a four-tuple (t, x, y, z)—where t is the *temporal coordinate*, representing

the *local time* at which the event takes place, and x, y, and z are the three *spatial coordinates*, representing the *local displacement* of the event in the reference frame R. The coordinates are *local* to R, in the sense that the values of these four coordinates for the same event e would be different in a different reference frame R'.

Chronological Orderings in Relativity. Since the temporal coordinate in a given reference frame is a real number, given a reference frame R, events can be totally ordered by the reference frame's *chronological order*, $<_R$. In particular, we say that $e <_R e'$, if $t_R(e) < t_R(e')$, i.e., if the temporal coordinate of e according to the reference frame R is less than the temporal coordinate of e' according to the same reference frame.

Relativity of Simultaneity. A particularly counterintuitive conclusion of relativistic physics is the *relativity of simultaneity*. This principle tells us that events in different reference frames not only have different time (and space) coordinates, but that the *chronological order* of events in different reference frames is also reference frame dependent. More concretely, relativity informs us that it is possible to have two events—e and e'—and three different reference frames—R, R^*, and R^{**}—such that the following three statements hold: (1) $e =_R e'$, (2) $e <_{R^*} e'$, and (3) $e >_{R^{**}} e'$. In words, events e and e' may be simultaneous in one reference frame, even as e chronologically precedes e' in another reference frame and e' chronologically precedes e in yet another reference frame.

The Relativistic Causal Ordering. While the relativity of simultaneity implies that events cannot be totally chronologically ordered in a manner consistent with all reference frames, relativistic physics does stipulate that spacetime events are, in general, partially ordered. In particular, the *relativistic (causal) order*, denoted $\prec$, is an irreflexive, transitive ordering of spacetime events. This relativistic order is *respected* by all chronological orders, meaning that if e relativistically precedes e', then e chronologically precedes e' in every reference frame (i.e., $e \prec e' \implies \forall R : e <_R e'$). The relativistic order has the following physical interpretation: $e \prec e'$ if and only if a hypothetical pulse of light originating at e could reach e'. The speed of light is a fundamental speed limit on all transmissions, so if even light could not reach e' from e, then e' could not possibly be effected by e, whereas if a light pulse could get from e to e', then e could possibly causally effect e'. In relativity, pointlike bodies trace out paths known as *world lines*, and all the spacetime events on a world line are totally ordered by the relativistic order.

We summarize the discussion with the following *relativistic postulates*:

RP1: Events of interest occur at distinct points in spacetime, called *(spacetime) events*, which are partially ordered by the irreflexive, transitive, relativistic causal precedence order, $\prec$. Every reference frame R's chronological total order of events, $<_R$, *refines* (i.e., respects) $\prec$.

RP2: Each pointlike body traces out a world line of spacetime events over its existence. Events on a world line are totally ordered by $\prec$.

RP3: Transmissions can travel up to the speed of light, but not faster. Thus, it is physically permissible for a transmission sent at e to be received at e' if and only if $e \prec e'$.

Remark 3.1. We have discussed the spacetime, $\mathcal{E}$, as a four-dimensional entity. Some treatments of relativity work in fewer or more dimensions. For example, Minkowski diagrams are drawn in two-dimensions, representing just one dimension of space and one dimension of time. Note that the relativistic postulates (RP1-RP3) are written in a dimension-independent form, and apply to spacetime models with fewer or more dimensions as well.

4 Distributed Algorithms: Classical and Relativistic Executions

We define distributed algorithms and computational steps in preparation for defining classical and relativistic executions.

Definition 4.1 (algorithm)**.** A *distributed algorithm*, $\mathcal{A}$, for a system of processors Π is a collection of algorithms $\mathcal{A} = \{\mathcal{A}_\pi\}_{\pi\in\Pi}$. We model processors as *state machines*, so an algorithm $\mathcal{A}_\pi$ specifies:

- a set of *states* Σ_π that processor π can be in,
- a set of possible *initial states* $\Sigma_\pi^0 \subseteq \Sigma_\pi$ for each processor $\pi \in \Pi$,
- a set E_π of *computational events* that can occur at processor π, and
- a *transition function*, $\delta_\pi : \Sigma_\pi \times E_\pi \times \Sigma_\pi \to \{true, false\}$. The value of the predicate $\delta_\pi(\sigma_\pi, e_\pi, \sigma'_\pi)$ is *true* if event e_π occurring at π can transition it from state σ_π to state σ'_π.

Two important classes of events in a distributed algorithm correspond to *message transmission* from some *universe of messages*, $\mathcal{M}$. We model the event of π sending the message m to π' by the event $(\pi, send, m, \pi')$ occurring at π. We say that π is the *sender* and that π' is the *intended recipient*. We model message reception by the event $(\pi', receive, m)$ occurring at π', which we call the *recipient*. We note that by a message, we mean an *instance* of a message, thus all messages are unique (even if their contents are identical).

Algorithms execute in steps, which are atomic units of computation that occur at processors.

Definition 4.2 (computational step)**.** A *step* of an algorithm $\mathcal{A}$ at a processor π is a quadruple $s_\pi = (\pi, \sigma_\pi, e_\pi, \sigma'_\pi)$ that satisfies π's transition function: $\mathcal{A}.\delta_\pi(\sigma_\pi, e_\pi, \sigma'_\pi)$. We say the step *occurs at* π, *starts at* $\sigma_\pi \in \mathcal{A}.\Sigma_\pi$, *ends at* $\sigma'_\pi \in \mathcal{A}.\Sigma_\pi$, and that the *event of the step* is $e(s_\pi) = e_\pi \in \mathcal{A}.E_\pi$. We denote the set of all steps that can occur at π by $\mathcal{S}_\pi$, and define $\mathcal{S} = \bigcup_{\pi\in\Pi} \mathcal{S}_\pi$.

Given a (finite or infinite) sequence of steps $\vec{s}_\pi = s_\pi^1, s_\pi^2, \ldots$ that all occur at π, we say the sequence *starts at* the state that the first step, s_π^1, starts at. If the sequence is finite, we say that it *ends at* the state that the last step in the sequence ends at. We say the sequence is *rooted* if it starts at an initial state (in $\mathcal{A}.\Sigma_\pi^0$), and *consecutively permissible* if each pair of consecutive steps (s_π^i, s_π^{i+1}) in the sequence have the property that s_π^{i+1} starts at the state in which s_π^i ends.

By our definition, a step is local to a processor, which accurately captures the physical nature of the transition event. It also avoids the need to define a global state of the underlying distributed system (in contrast to, e.g., [27]), which is critical, since in relativity different observers may have different views of the global state. Modeling steps locally can make reasoning about classical systems

slightly more challenging, but in return, the physical precision of the model allows the flexibility to analyze both classical and relativistic distributed systems in a unified computational framework.

The preceding definitions and discussion bring to light an important distinction between distributed systems and the distributed algorithms they run. Distributed systems are systems of physical processors that are embedded in an underlying physics—classical or relativistic. The processors occupy physical locations and may be in motion. In contrast, distributed algorithms can be described, independent of what type of physical system they may run on. The activity that brings together these physical and computational worlds is algorithmic execution. Thus, the definition of an *execution* is the glue between the physical model of the distributed system of processors and the computational model of a distributed algorithm that runs on it.

Definition 4.3 (physical executions: classical and relativistic)**.** A *physical execution* of a distributed algorithm $\mathcal{A}$ on a distributed system of processors Π is described by the following three principles:

- $\Phi 1$: Each processor $\pi \in \Pi$ is a pointlike body that starts in an initial state $\sigma_\pi^0 \in \mathcal{A}.\Sigma_\pi^0$. The execution consists of a set of steps $S \subseteq \mathcal{A}.\mathcal{S}$ that occur at the processors.
- $\Phi 2$: The steps $S_\pi \subseteq S$ that occur at a given processor π are physically permissible and in accordance with the transition function $\mathcal{A}.\delta_\pi$, meaning that a step $s_\pi = (\pi, \sigma_\pi, e_\pi, \sigma'_\pi)$ can occur at π only if its current state is σ_π, e_π occurs at π, and $\delta_\pi(\sigma_\pi, e_\pi, \sigma'_\pi)$ is true.
- $\Phi 3$: Message transmissions are *valid*, *physical* transmissions occurring with *at most once* semantics. At most once semantics means that a message that is sent is received at most once. Validity means that only the intended recipient of a message can receive the message.

The three principles stated above ensure that the physical postulates are applicable to physical executions. We model classical and relativistic physical executions below.

- A *classical physical execution* is a physical execution of the algorithm that is permissible under classical physics, i.e., it is a pair $\mathcal{X} = (S, t : S \to \mathbb{R})$, where S is a set of *steps*, and t is a *temporal mapping* that maps each step $s \in S$ to the distinct time $t(s)$ at which it occurs.
- A *relativistic physical execution* is a physical execution of the algorithm that is permissible under relativistic physics, i.e., it is a pair $\mathcal{Y} = (S, \varepsilon : S \to \mathcal{E})$, where S is a set of *steps*, and ε is a *spacetime mapping* that maps each step $s \in S$ to the distinct spacetime event $\varepsilon(s)$ at which it occurs.

The above definitions of physical executions capture the precise temporal (resp. spacetime) coordinates of computational steps in classical (resp. relativistic) physics. For several purposes, including defining and evaluating strong consistency, it is sufficient to capture just the ordering of the steps. To do so, we identify the computational steps with the point in time (resp. spacetime) at which they occur, and simply model the total (resp. partial) ordering of these steps. Doing so gives rise to the following definitions of classical and relativistic executions.

Definition 4.4 (classical and relativistic executions)**.**

- A *classical execution* is a pair $X = (S, <_S)$ which is *extracted* from a physical classical execution $\mathcal{X} = (S, t)$, in the sense that $<_S$ is a total order such that $s <_S s'$ if and only if $t(s) < t(s')$.
- A *relativistic execution* is a pair $Y = (S, \prec_S)$ which is *extracted* from a physical relativistic execution $\mathcal{Y} = (S, \varepsilon)$, in the sense that $\prec_S$ is a partial order such that $s \prec_S s'$ if and only if $\varepsilon(s) \prec \varepsilon(s')$.

We assume a slight abuse of notation that allows events of steps to be compared by $<_S$ and $\prec_S$.

5 Shared Objects, Linearizability, and Relativistic Linearizability

A shared object is specified by the *sequential specification* of its *type*, $\mathcal{T}$, which determines the valid behaviors of the object when there are no concurrent operations [15].

Definition 5.1 (distributed implementation)**.** A *distributed implementation of a shared object* is a distributed algorithm $\mathcal{A}$, where each processor π can non-deterministically choose to invoke an operation op by the *invocation event* $(\pi, invoke, op)$, and return a response r to its invocation by the *response event* $(\pi, respond, op, r)$ occurring at π. Just as with messages, here op symbolizes a unique *instance* of an operation.

Definition 5.2 (linearizability)**.** We give the formal definition of *linearizability* in Appendix A. Informally, a distributed algorithm $\mathcal{A}$ is linearizable if all operations of classical executions appear to take effect instantaneously at some time between their invocation and response.

Definition 5.3 (relativistic linearizability)**.** A relativistic execution is *linearizable in a reference frame* R, if the total ordering of the execution according to the local time of reference frame R is linearizable. A relativistic execution is *relativistically linearizable*, if it is linearizble in every reference frame. A distributed algorithm $\mathcal{A}$ is *relativistically linearizable*, if every relativistic execution of the algorithm is relativistically linearizable.

6 Relating Classical and Relativistic Executions

A central tool in relating the properties of classical and relativistic executions will be a third kind of execution, called a computational execution.

6.1 Computational Execution

The goal behind defining computational executions is to abstract away from the physics and capture only the essential orderings between the steps. Our intuition is that classical and relativistic executions of algorithms carry a lot of ordering information between steps that is purely an artifact of the underlying physics, but irrelevant to the algorithm. In order to shed these extra orderings, we appeal to a classic tool in distributed computing, *computational causality*, i.e. Lamport's *happens-before*, relation [24]. Informally, a *computational execution* of a distributed algorithm is one that captures only the steps of the algorithm and the computational (causal) order between them. Computational executions

are a partially ordered collection of steps, just as relativistic executions are, thus *computational linearizability* is naturally defined in analogy.

Definition 6.1 (computational executions and computational linearizability)**.**

- A *computational execution* of an algorithm $\mathcal{A}$ is a pair $Z = (S, \lhd_S)$, satisfying the axioms:
  - A1: S is a set of computational steps of $\mathcal{A}$, and $\lhd_S$ is a relation over S that is the transitive closure of the relation satisfying the following axioms.
  - A2: For every process $\pi \in \Pi$, the subset of steps $S_\pi \subseteq S$ that are performed at π form a rooted and consecutively permissible totally ordered sequence under $\lhd_S$.
  - A3: Message transmissions are *valid*, satisfy *at most once* semantics, and are *oriented forward* in $\lhd_S$. Valid means that a message is only received by the intended recipient; at most once semantics means that a message that is sent is received at most once; and, oriented forward means that if m is sent in step s and received in step s', then $s \lhd_S s'$.

  We call the relation $\lhd_S$ the *computational causality* relation.
- A computational execution is *computationally linearizable* if every total ordering of it is linearizable, and an algorithm $\mathcal{A}$ is *computationally linearizable* if every computational execution of the algorithm is computationally linearizable.

It is noteworthy that both computational executions and computational linearizability are physics-independent, purely mathematical notions.

6.2 Relating Classical, Relativistic, and Computational Executions

Computational executions serve as the bedrock of our analysis. Intuitively, the computational executions of an algorithm are defined to carry only the necessary relationships between steps needed to form a valid execution of the algorithm. Our goal in the next two lemmas will be to formalize this intuition as it concerns classical and relativistic executions. In particular, we will show that *every relativistic execution $Y = (S, \prec_S)$ is a* refinement *of some computational execution $Z = (S, \lhd_S)$*. In the case of classical executions, we can make an even stronger statement, which characterizes classical executions in terms of computational executions. In particular, we will show that *the set of all classical executions of an algorithm is exactly equal to the set of all total orderings of computational executions of that algorithm.* In this context, *$\prec_S$ refines $\lhd_S$* means that $\prec_S \supseteq \lhd_S$, and *$<_S$ is a total ordering* of $\lhd_S$ means that $<_S$ is a total order which refines $\lhd_S$.

Lemma 6.2. *Every relativistic execution, $Y = (S, \prec_S)$, of a distributed algorithm $\mathcal{A}$ is a refinement of some computational execution, $Z = (S, \lhd_S)$, of $\mathcal{A}$.*

Proof. Given the relativistic execution, $Y = (S, \prec_S)$, we first formally define the pair $Z = (S, \lhd_S)$ by defining the relation $\lhd_S$. Our aim is to capture only the relationships between computationally causally related steps of S in the execution Y, so we focus on steps that occur at the same processor and pairs of steps that correspond to the sending and reception of a message.

Let $\lhd_S$ be the transitive closure of the relation on S that satisfies:

(a) For each processor π and each pair of steps $s, s' \in S$ that occur at π, let $s \lhd_S s'$ if $s \prec_S s'$.
(b) For every message m and pair of steps $s, s' \in S$ such that m is sent in step s and received in step s', let $s \lhd_S s'$.

First we prove that $Z = (S, \lhd_S)$ is a computational execution by showing that it satisfies the three axioms of Definition 6.1.

A2: We consider an arbitrary processor π and consider the subset of steps $S_\pi \subseteq S$ that occur at that processor. By principle $\Phi 2$ of physical executions, these steps occur at a physical pointlike processor π. Thus, by postulate RP2 of relativistic physics, these steps are totally ordered by $\prec_S$ into a sequence $\vec{s}_\pi = s^1_\pi, s^2_\pi, \ldots$. By principle $\Phi 1$ of physical executions, processor π starts in an initial state σ^0_π, thus the sequence $\vec{s}_\pi$ must be rooted. By principle $\Phi 2$, steps must be in accordance with the transition function $\mathcal{A}.\delta_\pi$, thus $\vec{s}_\pi$ must be consecutively permissible. Thus, the steps in S_π are rooted, consecutively permissible, and totally ordered by $\lhd_S$, thereby proving A2.

A3: Physical execution principle $\Phi 3$ yields that message transmissions are valid and satisfy at most once semantics in Y. Since these are really properties of the set of steps S, they are inherited by Z. Message transmissions are sent forward in Z by item (b) of the definition.

A1: The challenge is in proving that the relation $\lhd_S$, which is defined as the transitive closure of relationships defined in items (a) and (b), is precisely equivalent to the relation $\lhd$ that axiom A1 specifies, which is defined as the transitive closure of the relationships defined in axioms A2 and A3. Since A2 and A3 are satisfied, it is clear that $\lhd_S \supseteq \lhd$. To show that there are no extra relationships in $\lhd_S$ (that are not in $\lhd$), we now argue that item (a) only adds relationships between steps that axiom A2 does. (It is clear that item (b) only adds the relationships that axiom A3 does.) To this effect, it suffices to see that item (a) only adds relationships between steps occurring at the same process; since A2 totally orders steps occurring at the same process, the proof is complete.

Next we prove that Y refines Z. In particular, we must show that every pair of steps $s, s' \in S$ that are related by $\lhd_S$, are related by $\prec_S$. Since, $\prec_S$ is transitive by postulate RP1, and $\lhd_S$ is a transitive closure, it suffices to show that the pairs of steps that are related in $\lhd_S$ by items (a) and (b) are related by $\prec_S$. This is self evident for the steps related by item (a), since item (a) only says that certain steps that are related by $\prec_S$ must also be related by $\lhd_S$. As for item (b), principle $\Phi 3$ of physical executions insists that message transmissions are physical transmissions, and postulate RP3 guarantees that the sending of a transmission relativistically precedes its reception. That completes the proof. □

Lemma 6.3. *The following is a characterization of classical executions of a distributed algorithm $\mathcal{A}$. Any classical execution, $X = (S, <_S)$, satisfies the following three properties:*

P1: S is a set of steps of $\mathcal{A}$, and $<_S$ is a total ordering of these steps.

P2: For each process $\pi \in \Pi$, the subset of steps $S_\pi \subseteq S$ that occur at π form a rooted, consecutively permissible sequence.
P3: Message transmissions in S are valid, satisfy at most once semantics, and are oriented forward by $<_S$.

Furthermore, any pair $X = (S, <_S)$ that satisfies P1, P2, and P3, is a classical execution.

Proof. For the forward direction, consider an arbitrary classical execution $X = (S, <_S)$. We prove that X satisfies the three properties:

P1: That S is a set of steps of $\mathcal{A}$ is guaranteed by principle $\Phi 1$, and that these steps are totally ordered by $<_S$ is guaranteed by postulate CP1. That proves property P1.
P2: We consider an arbitrary processor π and consider the subset of steps $S_\pi \subseteq S$ that occur at that processor. By postulate CP1 these steps can be ordered into a sequence $\vec{s}_\pi = s^1_\pi, s^2_\pi, ...$ By principle $\Phi 1$ of physical executions, processor π starts in an initial state σ^0_π, thus the sequence $\vec{s}_\pi$ is rooted. By principle $\Phi 2$, steps must be in accordance with the transition function $\mathcal{A}.\delta_\pi$, thus $\vec{s}_\pi$ must be consecutively permissible.
P3: Message transmissions are valid and satisfy at most once semantics by principle $\Phi 3$ of physical executions. The same principle guarantees that message transmissions are physical transmissions, whereby the time that any message is received must succeed the time at which it is sent by postulate CP2. Thus, messages are oriented forward.

Now, consider an arbitrary pair $X = (S, <_S)$ that satisfies the three properties, and construct a temporal mapping $t : S \to \mathbb{R}$ of the steps as follows. Letting $\vec{s} = s^1, s^2, ...$ be the sequence of steps in S ordered by $<_S$, define $t(s^i) = i$, indicating that the ith step is performed at time i.

We now argue that $\mathcal{X} = (S, t : S \to \mathbb{R})$ is a classical physical execution from which X can be extracted to finish the proof. To show that $\mathcal{X}$ is a classical physical execution, we must show that it satisfies principles $\Phi 1$, $\Phi 2$, and $\Phi 3$ and is permissible under classical physics. By P1 and P2, S consists of steps from algorithm $\mathcal{A}$ and the subset of steps $S_\pi \subseteq S$ occurring at any given processor π form a rooted, consecutively permissible sequence $\vec{s}_\pi$. By the definition of rooted, each processor π starts at an initial state σ^0_π which gives us $\Phi 1$. Since the steps occur at distinct times by t, they are physically permissible under postulate CP1, thus satisfying $\Phi 2$. Messages are valid and satisfy at most once semantics, thus to show $\Phi 3$, we must show that the messages are physically permissible. Since messages are oriented forward by $<_S$, and the temporal mapping t orders steps in the same way as $<_S$, postulate CP2 says that the message transmissions are physically permissible. Thus, $\mathcal{X}$ is a classical physical execution. X can be extracted from $\mathcal{X}$, so that finishes the proof. □

Lemma 6.4. *Every classical execution of a distributed algorithm is a total ordering of a computational execution of the same algorithm, and every total ordering of a computational execution of a distributed algorithm is a classical execution of the same algorithm.*

Proof. The proof relates classical and computational executions by appropriately relating the properties P1, P2, and P3 of Lemma 6.3 with the axioms A1, A2, and A3 of Definition 6.1.

To show that every classical execution is a total ordering of a computational execution, consider an arbitrary classical execution, $X = (S, <_S)$ of a distributed algorithm $\mathcal{A}$. We formally define a pair $Z = (S, \lhd_S)$ by defining the relation $\lhd_S$; we will later show that it is a computational execution and that X refines it. Let $\lhd_S$ be the transitive closure of the relation on S that satisfies:

(a) For each processor π and each pair of steps $s, s' \in S$ that occur at π, let $s \lhd_S s'$ if $s <_S s'$.
(b) For every message m and pair of steps $s, s' \in S$ such that m is sent in step s and received in step s', let $s \lhd_S s'$.

We first prove that Z is a computational execution by showing that it satisfies the three axioms A1, A2, and A3. We know that X satisfies the properties P1, P2, and P3 of Lemma 6.3. Property A2 of Z follows from property P2 of X and item (a). Property A3 of Z follows from property P3 of X and item (b). We now turn to property A1 of Z. Property P1 of X implies that S is a collection of algorithmic steps of $\mathcal{A}$; but it remains to be shown that $\lhd_S$ is equivalent to the transitive closure, $\lhd$, of the relationships defined by axioms A2 and A3. To this effect, we note that item (a) defines precisely the relationships specified by A2 and item (b) defines precisely the relationships specified by axiom A3, thus giving us property A1.

To show that every total ordering of a computational execution is a classical execution, we consider an arbitrary computational execution, $Z = (S, \lhd_S)$ of a distributed algorithm $\mathcal{A}$, and consider any total ordering $X = (S, <_S)$ of it. To complete the proof, we must show that X is a classical execution; we do this below by proving that it satisfies the three properties. S is a set of computational steps by axiom A1 of Z and they are totally ordered by $<_S$ by the definition of a total ordering, yielding property P1 of X. Property P2 of X follows from axiom A2 of Z, since $<_S$ is a total ordering of $\lhd_S$. Finally, property P3 of X follows from axiom A3 of Z, once again, since $<_S$ is a total ordering of $\lhd_S$. That concludes the proof by Lemma 6.3. □

Corollary 6.5. *Every total ordering of a relativistic execution of a distributed algorithm is a classical execution of the same algorithm.*

Proof. Lemma 6.2 shows that every relativistic execution Y is a refinement of a computational execution Z. Thus, any total ordering of Y is also a total ordering of Z, and thereby a classical execution by Lemma 6.4. □

We now relate classical and relativistic executions directly, by arguing that every classical execution is also a relativistic execution. Naturally, the argument is heavily based on the postulates of physics.

Lemma 6.6. *Every classical execution of a distributed algorithm is also a relativistic execution of the same algorithm.*

Proof. Consider a distributed algorithm $\mathcal{A}$ and fix a classical execution $X = (S, <_S)$ of the algorithm. By definition, X is extracted from a classical physical execution $\mathcal{X} = (S, t : S \to \mathbb{R})$. Our goal is to show that X can also be extracted from a relativistic

physical execution, since that would imply that X is also a relativistic execution.

We now define a relativistic physical execution $\mathcal{Y} = (S, \varepsilon : S \to \mathcal{E})$ based on $\mathcal{X}$. The key physical insight in our argument is to consider a relativistic distributed system in which all the processors of Π coincide, such that they trace out the same world line $\mathcal{L} \subseteq \mathcal{E}$ of spacetime events. All the computational events in our execution will occur at processors that trace out the same world line, thus, postulate RP2 says that all the events will be totally ordered by the relativistic order. Since all the processors coincide, they share a reference frame; let us call that reference frame R. By postulate RP1, the chronological total order on spacetime events of $\mathcal{L}$ according to reference frame R must coincide with the relativistic total order of these spacetime events, so each time value, τ, in the reference frame R corresponds to a unique spacetime event $e(\tau) \in \mathcal{L}$ on the world line. We fix $\mathcal{Y}$ by defining the mapping $\varepsilon(s \in S) = e(t(s))$. Since $\mathcal{X}$ ensures that steps occur at distinct times (by CP1), our mapping ensures that computational events occur at distinct spacetime events in $\mathcal{Y}$ (satisfying RP1). Since all events are totally ordered, RP2 is satisfied. Since $\mathcal{X}$ ensures that message transmissions are oriented forward in time (by CP2), our mapping ensures that message transmissions are oriented forward according to the relativistic relation in $\mathcal{Y}$ (satisfying RP3). Thus, $\mathcal{Y}$ is physically permissible in relativistic physics and the three principles of physical executions (Φ1, Φ2, and Φ3) coincide for $\mathcal{X}$ and $\mathcal{Y}$. Finally, since $\prec$ totally orders the steps of S just as $<_S$ does, X can be extracted from $\mathcal{Y}$, thereby proving that X is a relativistic execution. □

Using the preceding lemmas, we prove a central relationship between properties of classical, relativistic, and computational executions.

Theorem 6.7 (Central Theorem)**.** *Let $\mathcal{A}$ be any distributed algorithm, and let $\mathcal{P}$ be any property, i.e., a predicate on classical executions. Then the following three statements are equivalent: (1) $\mathcal{P}$ is satisfied by all classical executions of $\mathcal{A}$; (2) $\mathcal{P}$ is satisfied by all total orderings of computational executions of $\mathcal{A}$; and (3) $\mathcal{P}$ is satisfied in the reference frame of every observer of every relativistic execution of $\mathcal{A}$.*

Proof. We prove the equivalence by the circle of implications: (1) $\implies$ (2) $\implies$ (3) $\implies$ (1). For (1) $\implies$ (2): we note that all total orderings of computational executions are classical executions by Lemma 6.4. For (2) $\implies$ (3): we note that all total orderings of relativistic executions are total orderings of computational executions, since a relativistic execution is a refinement of a computational execution by Lemma 6.2. For (3) $\implies$ (1): we note that all classical executions are totally ordered and are relativistic executions by Lemma 6.6. □

As a consequence of the Central Theorem, we derive the equivalence of classical, relativistic, and computational linearizability.

Theorem 6.8 (Equivalence Theorem)**.** *Let $\mathcal{A}$ be any distributed implementation of a shared object. Then, the following three statements are equivalent: (1) $\mathcal{A}$ is linearizable; (2) $\mathcal{A}$ is computationally linearizable; and (3) $\mathcal{A}$ is relativistically linearizable.*

Proof. We instantiate Theorem 6.7 with the property $\mathcal{P}$ which is satisfied if a classical execution is linearizable. Thus instantiated, the three statements of that theorem becomes: (1) $\mathcal{A}$ is linearizable; (2) $\mathcal{A}$ is computationally linearizable; and (3) $\mathcal{A}$ is relativistically linearizable; thus concluding the proof. □

7 A Quirk of Relativistic Linearizability

In this section, we illustrate the curious nature of relativistic executions by examining the Jayanti-Tarjan Union-Find (JTUF) object [21–23]—a linearizable concurrent algorithm for the *disjoint set union* data type. In particular, we demonstrate the following intriguing property of relativistic linearizability: when a linearizable object is deployed in a relativistic distributed system, all observers will agree that the execution is linearizable (by the Equivalence Theorem), however, they *may have to* disagree on the order in which the operations linearize.

7.1 The JTUF algorithm

The JTUF algorithm is a machine-verified [20] linearizable implementation of the *disjoint set union* (a.k.a. *union-find* or *set union*) data type that is used extensively in practice, including in graph analysis [9], large-scale clustering [14], and model checking [7]. The object maintains a *partition* of n *nodes* in disjoint *blocks*. The object supports the operations:

- Unite(x, y): coarsens the partition by combining the blocks containing nodes x and y, if x and y are in different blocks of the partition.
- SameSet(x, y): returns whether x and y are in the same block.

JTUF implements these operations using a compressed tree representation in which each node stores a single memory word, called a *parent pointer*. The details of the implementation are irrelevant to our current discussion (the curious reader can consult [23]). For our purpose, it suffices to note that nodes are stored at individual processors, and that if all of the nodes in x's block and all of the nodes in y's block are stored at a single processor π, then the algorithm ensures that both the operations—Unite(x, y) and SameSet(x, y)—can be performed at π without sending or receiving any messages (regardless of whether nodes associated with other blocks are stored at different processors).

7.2 Deployment in a relativistic setting

Since JTUF is linearizable, all observers will perceive every relativistic execution of JTUF to be linearizable by the Equivalence Theorem (Theorem 6.8). Nevertheless, through an Einsteinian physical thought experiments, we will demonstrate that in some relativistic executions of JTUF, there are a pair of observers who *necessarily* disagree on the order in which operations linearize.

The crux of the physical setup comes straight from Einstein's thought experiment on special relativity (of high-speed trains passing by an embankment that is struck by lightning) [11]. In the discussion below, we will replace his trains with spaceships and lightning-strike events with computational events.

Consider the physical setup visualized in Figure 1. Here, we have a relativistic distributed system consisting of two processors $\Pi = \{\pi_1, \pi_2\}$. The processors are at rest with respect to the

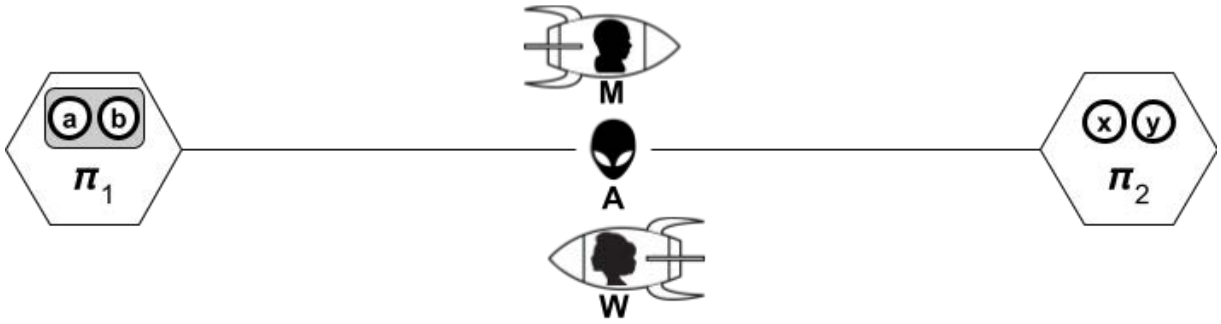


Figure 1: A relativistic distributed system consisting of two processors $\Pi = \{\pi_1, \pi_2\}$ which are stationary in the reference frame of the alien situated at their midpoint A. The system maintains a JTUF object with $n = 4$ nodes, where nodes a and b are located at π_1 and nodes x and y are located at π_2, and the blocks of the partition are $\{a, b\}$, $\{x\}$, and $\{y\}$. A man and a woman travel at high speeds and in opposite directions, passing by this setup in their individual spaceships. The man sits at the point M of his spaceship and traveling left-to-right, and the woman sits at the point W of her spaceship and traveling right-to-left.

observer—whom we will call the *alien*—that is sitting at their midpoint A. The processors are also maintaining a JTUF object with $n = 4$ nodes, where nodes a and b are located at π_1 and nodes x and y are located at π_2, and the blocks of the partition are $\{a, b\}$, $\{x\}$, and $\{y\}$. Two other observers, a man and a woman, are traveling at high speeds and in opposite directions, passing by this setup in their individual spaceships. The man is sitting in the middle M of his spaceship and traveling left-to-right, while the woman is sitting in the middle W of her spaceship and traveling right-to-left. In our thought experiment, the man and the woman coincidentally pass by the alien, such that there is a spacetime event e which corresponds to the spacetime at which all three entities intersect. Furthermore, we imagine that the following takes place:

- π_1 invokes SameSet(a, b) on the JTUF object, and receives the response *true* for this operation. Since the computation associated with this operation is performed entirely at the processor π_1 with no communication, we consider the case in which the set of computational events E_1 that comprise this computation all lie within a very small ϵ-ball around a spacetime event e_1. Informally speaking, the entire computation happens almost instantaneously at a spacetime event e_1.
- π_2 invokes SameSet(x, y) on the JTUF object and receives the response *false*; once again we consider the case in which the entire operation is almost instantaneous at spacetime event e_2. More precisely, the computational events of the operation, E_2, all lie within a very small ϵ-ball around a spacetime event e_2.
- In the alien's reference frame all three events—e, e_1, and e_2—happen at the same time.

Let $<_M$, $<_W$, and $<_A$ be the chronological order of events in the reference frames of the man, woman, and alien, respectively. Einstein's derivations (through the Lorentz transform of special relativity) imply that for a sufficiently small ϵ, even though $e_1 =_A e_2$, we have $e_2' <_M e_1'$ and $e_1' <_W e_2'$ for *every* $e_1' \in E_1$ and $e_2' \in E_2$. In particular, this means that in the local time of the man's reference frame the operation SameSet(x, y) ends before SameSet(a, b) starts, while in the local time of the woman's reference frame SameSet(a, b) ends before SameSet(x, y) starts. Thus, it is impossible for the man and the woman to find linearizations of the execution in their individual reference frames such that they agree on order in which the operations linearize!

We summarize the discussion in the following observation.

Observation 1. *There is a relativistic execution of an algorithm using a relativistically linearizable object in which two observers necessarily disagree on the order in which the operations linearize.*

Remark 7.1. Note that Observation 1 does not contradict the fact that both observers perceive the object to be linearizable, since the operations that are reordered commute.

8 Final Remarks

Having mapped out our own planet, human civilization is eager to explore new frontiers. Relativistic distributed systems of satellites, spaceships, telescopes, and planetary rovers that venture past the horizons of the known and probe the expanse of our solar system and beyond are likely to be the critical infrastructure that enables this exploration. Simultaneously, as the Global Positioning System (GPS) reminds us, relativistic distributed computation is not just a futuristic dream, but a crucial mainstay of our everyday lives. Furthermore, relativistic computing infrastructure is burgeoning with advances in the Interplanetary Internet.

In this paper, we have studied the nature of computation on relativistic distributed systems, asking ourselves how distributed algorithms—which have largely been designed for systems embedded in classical physics—would behave when deployed in relativistic distributed systems. Due to the partially ordered nature of events in relativistic spacetime, this exploration also led us to examine the structure of the underlying computational relationships between steps in an executing algorithm through the mathematical notion of computational executions. We expect that this notion of computational executions will find widespread use in the analysis of both classical and relativistic distributed computing.

Our exploration led to two important technical theorems that relate properties of classical, relativistic, and computational executions of distributed algorithms. Through our Central Theorem

(Theorem 6.7), we proved that properties that are satisfied by classical executions of a distributed algorithm will be satisfied in every reference frame, if the algorithm is executed in a relativistic distributed system. In fact, we showed that for *any* distributed algorithm $\mathcal{A}$, *any* property $\mathcal{P}$ is satisfied by all classical executions *if and only if* it is satisfied in all reference frames of all relativistic executions. We expect that the Central Theorem is likely to have widespread use in reasoning about relativistic distributed systems, due to its generality. As a consequence of this general theorem, we proved the Equivalence Theorem (Theorem 6.8): that an algorithm is linearizable *if and only if* it is computationally linearizable *if and only if* it is relativistically linearizable. Practically, our equivalence theorem guarantees that the host of linearizable asynchronous algorithms that the distributed computing community has discovered for classical systems will remain strongly consistent when deployed in relativistic, interplanetary distributed systems.

A Linearizability Definitions

Informally, an execution is *linearizable*, if operations—which actually take place over intervals of time—appear to take place instantaneously in their interval of execution. To formalize this definition, it turns out we must reason not only about orders of *events* but also about the orders of *operations*. We first define *histories*, then we define Lamport's *happens-before* partial ordering over operations [25], and finally define linearizability.

Definition A.1 (classical history)**.** A *classical history* is a pair $H = (E, <_E)$, where E is a set of invocation and response events and $<_E$ is a total ordering of them. A history is well formed if all response events $e_r = (\pi, respond, op, r)$ in E *match* some invocation $e_i = (\pi', invoke, op')$ in E, in the sense that $\pi' = \pi$, $op' = op$, and $e_i <_E e_r$. We call an operation that has an invocation but no matching response, *pending*.

We can *extend* a history $H = (E, <_E)$ by adding some matching responses to pending operations. That is, a history $\overline{H} = (\overline{E} = E \cup R, <_{\overline{E}})$, where R is a set of matching responses to pending operations in E, and $<_{\overline{E}} = <_E \cup (E \times R)$ an *extension* of H. A *complete* history is one in which every invocation has a matching response.

A history is *sequential* if it is well formed and is an alternating sequence of invocations and matching responses. A sequential history is *legal* for a type $\mathcal{T}$ if the responses conform to the sequential specification of the type.

Given a classical execution $X = (S, <_S)$, its *history*, $(E, <_E)$, is the set E of invocation and response events in X and their induced ordering $<_E$.

Definition A.2 (operation executions and happens-before)**.** Given a history $H = (E, <_E)$, we can consider its *operation executions (opex)*. An opex of an operation op that is invoked in H, denoted $ox(op)$, is a set comprising of op's invocation event and—if op has a matching response event in the history—op's matching response event. Given two opexs ox and ox', we say ox *happens before* ox', denoted $ox \longrightarrow_H ox'$, if the response of ox exists and occurs before the invocation of ox'.

Definition A.3 (linearizability)**.** A history H is *linearizable*, if: (1) H is well formed; (2) there is a complete extension $\overline{H} = (E, <_{\overline{H}})$ of H and a sequential history $H' = (E, <_{H'})$ over the same events, such that the total ordering $\longrightarrow_{H'}$ respects the partial ordering $\longrightarrow_{\overline{H}}$. A distributed implementation $\mathcal{A}$ of a shared object is *(classically) linearizable*, if the history of every finite classical execution of it is linearizable.

Definition A.4 (relativistic linearizability)**.** A distributed algorithm $\mathcal{A}$ for a shared object is *relativistically linearizable*, if every relativistic execution of the algorithm is linearizable in the reference frame of every observer.

References

[1] Aguilera, M. K., and Frølund, S. Strict linearizability and the power of aborting. Tech. Rep. HPL-2003-241, Hewlett-Packard Labs, 2003.

[2] Ahamad, M., Neiger, G., Burns, J. E., Kohli, P., and Hutto, P. W. Causal memory: Definitions, implementation, and programming. *Distributed Computing 9*, 1 (1995), 37–49.

[3] Ashby, N. Relativity in the global positioning system. *Living Reviews in relativity 6* (2003), 1–42.

[4] Attiya, H., Ben-Baruch, O., and Hendler, D. Nesting-safe recoverable linearizability: Modular constructions for non-volatile memory. In *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing* (New York, NY, USA, 2018), PODC '18, Association for Computing Machinery, p. 7–16.

[5] Ben-David, N., Blelloch, G. E., Fatourou, P., Ruppert, E., Sun, Y., and Wei, Y. Constant-time lazy snapshots supporting general queries on concurrent data structures. *CoRR abs/2007.02372* (2020).

[6] Berryhill, R., Golab, W. M., and Tripunitara, M. Robust shared objects for non-volatile main memory. In *19th International Conference on Principles of Distributed Systems, OPODIS 2015, December 14-17, 2015, Rennes, France* (2015), E. Anceaume, C. Cachin, and M. G. Potop-Butucaru, Eds., vol. 46 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, pp. 20:1–20:17.

[7] Bloemen, V., Laarman, A., and van de Pol, J. Multi-core on-the-fly scc decomposition. In *Proceedings of the 21st ACM SIGPLAN symposium on Principles and practice of parallel programming* (2016), PPoPP '16, p. to appear.

[8] BlueOrigin. https://www.https://www.blueorigin.com/about-blue.

[9] Dhulipala, L., Hong, C., and Shun, J. Connectit: A framework for static and incremental parallel graph connectivity algorithms, 2020.

[10] Einstein, A. *The Meaning of Relativity.* Routledge, London, 1922.

[11] Einstein, A. *Relativity: The Special and General Theory.* Routledge, 1952.

[12] Gilbert, S., and Golab, W. M. Making sense of relativistic distributed systems. In *Distributed Computing - 28th International Symposium, DISC 2014, Austin, TX, USA, October 12-15, 2014. Proceedings* (2014), F. Kuhn, Ed., vol. 8784 of *Lecture Notes in Computer Science*, Springer, pp. 361–375.

[13] Golab, W. M., Higham, L., and Woelfel, P. Linearizable implementations do not suffice for randomized distributed computation. In *Proceedings of the 43rd ACM Symposium on Theory of Computing, STOC 2011, San Jose, CA, USA, 6-8 June 2011* (2011), L. Fortnow and S. P. Vadhan, Eds., ACM, pp. 373–382.

[14] Google. Graph-mining. `https://github.com/google/graph-mining`, 2023.

[15] Herlihy, M. P., and Wing, J. M. Linearizability: A correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst. 12*, 3 (July 1990), 463–492.

[16] Hooke, A. The interplanetary internet. *Commun. ACM 44*, 9 (Sept. 2001), 38–40.

[17] Howard, P. W. *Extending relativistic programming to multiple writers.* Portland State University, 2012.

[18] Izraelevitz, J., Mendes, H., and Scott, M. L. Linearizability of persistent memory objects under a full-system-crash failure model. In *Distributed Computing - 30th International Symposium, DISC 2016, Paris, France, September 27-29, 2016. Proceedings* (2016), C. Gavoille and D. Ilcinkas, Eds., vol. 9888 of *Lecture Notes in Computer Science*, Springer, pp. 313–327.

[19] Jayanti, P., Jayanti, S., and Jayanti, S. Durable algorithms for writable LL/SC and CAS with dynamic joining. In *37th International Symposium on Distributed Computing, DISC 2023, October 10-12, 2023, L'Aquila, Italy* (2023), R. Oshman, Ed., vol. 281 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, pp. 25:1–25:20.

[20] Jayanti, P., Jayanti, S., Yavuz, U., and Hernandez, L. A universal, sound, and complete forward reasoning technique for machine-verified proofs of linearizability. *Proc. ACM Program. Lang. 8*, POPL (jan 2024).

[21] Jayanti, S., Tarjan, R. E., and Boix-Adserà, E. Randomized concurrent set union and generalized wake-up. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing* (New York, NY, USA, 2019), PODC '19, Association for Computing Machinery, p. 187–196.

[22] Jayanti, S. V., and Tarjan, R. E. A randomized concurrent algorithm for disjoint set union. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing* (New York, NY, USA, 2016), PODC '16, ACM, pp. 75–82.

[23] Jayanti, S. V., and Tarjan, R. E. Concurrent disjoint set union. *Distributed Comput. 34*, 6 (2021), 413–436.
[24] Lamport, L. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM 21*, 7 (1978), 558–565.
[25] Lamport, L. On interprocess communication. part I: basic formalism. *Distributed Comput. 1*, 2 (1986), 77–85.
[26] Lamport, L. On interprocess communication. part II: algorithms. *Distributed Comput. 1*, 2 (1986), 86–101.
[27] Lynch, N. A. *Distributed Algorithms*. Morgan Kaufmann, 1996.
[28] MarsOne. Human settlement on mars. https://web.archive.org/web/20190131065806/https://www.mars-one.com/.
[29] SpaceX. Human space flight. https://www.spacex.com/humanspaceflight/.
[30] Triplett, J., McKenney, P. E., and Walpole, J. Scalable concurrent hash tables via relativistic programming. *ACM SIGOPS Operating Systems Review 44*, 3 (2010), 102–109.